

Systems Engineering and Integration of Counter-Swarm Defence Decision-Support Platforms

A structured research monograph on mission definition, interface control, hybrid edge–central architectures, digital twins, failure engineering, verification and validation, and staged integration

Author: Victor.I

Programme: Counter-Swarm Defence research monograph series

Domain: Systems engineering and integration (defensive decision support only)

Integrity rule: Citations refer to real standards and landmark works; no fabricated DOIs

Abstract

Counter-swarm defence decision-support platforms sit at the intersection of multi-sensor command-and-control (C2), software-intensive systems engineering, and safety-constrained human–machine teaming. The engineering problem is not “detect drones and shoot.” It is to assemble a timely, evidence-backed air picture from heterogeneous, late, missing, and contradictory observations; to support graded risk assessment under uncertainty; and to capture human-authorized response *categories* without autonomous harmful actuation. This monograph develops a systems engineering and integration framework for that class of platform, grounded in the lifecycle practices of ISO/IEC/IEEE 15288 and the process guidance of the INCOSE *Systems Engineering Handbook*, and specialised to a hybrid edge–central topology (x86 central operations node, NVIDIA Jetson-class electro-optical edge inference, Raspberry Pi only for light ingest), a digital-twin-first verification strategy preceding hardware-in-the-loop (HIL), interface control documentation (ICD) as the primary integration contract, and failure mode and effects analysis (FMEA) as a living safety and operability artefact.

The contribution is integrative rather than algorithmic. The monograph (1) restates the mission and MoSCoW boundaries that keep the platform inside defensive decision support; (2) shows how ICD-driven modular services avoid vendor lock-in and silent safety bypass; (3) justifies hybrid compute placement against bandwidth, latency, and fleet-operations trade-offs; (4) argues that a digital twin exercising the same adapter contracts as live sensors is a necessary precondition for credible HIL and field stages; (5) elaborates failure engineering and graceful degradation modes suited to sensor silence, fusion error, model drift, and audit failure; (6) defines verification and validation (V&V) evidence that can actually prove the system rather than merely exercise demos; and (7) maps a Stage 0–8 roadmap from research approval through controlled field testing. Throughout, trade-offs, failure

modes, and “what goes wrong in practice” are treated as first-class design content. Scope is explicitly defensive: sensing, fusion, tracking, risk, human approval, audit, and category-only external handoff. Weapon guidance, kinetic fire control, and electronic-attack execution recipes are out of scope.

Keywords: systems engineering; ISO/IEC/IEEE 15288; interface control; digital twin; FMEA; hybrid edge–central; counter-UAS; decision support; V&V; MoSCoW

Table of contents

1. Introduction and problem framing
 2. Related standards and literature
 3. Mission definition and operational concept
 4. Requirements engineering and MoSCoW
 5. Architectural approaches and justified selection
 6. Interface control documentation
 7. Hybrid edge–central compute integration
 8. Digital twin before hardware-in-the-loop
 9. Failure engineering and FMEA
 10. Verification, validation, and proof
 11. Roadmap stages 0–8
 12. Build and integration guide
 13. Pre-ship gate and ownership conclusions
 14. References
 15. Appendices (A–W): glossary, indexes, stage-exit templates, worked lab narrative, quality-attribute scenarios, RACI, time/association, edge fleet ops, audit-as-safety, 15288 mapping, demo-versus-good, risk register, document control, configuration baselines, CONOPS modes, integration anti-patterns, assurance claims, scaling notes, security co-ownership, maintenance/disposal, onboarding summary
-

1. Introduction and problem framing

1.1 Restatement of the problem

A defender facing possible coordinated unmanned aerial system (UAS) activity needs more than a camera feed or a radar plot. They need a coherent operational picture: which objects exist, where they are with what uncertainty, which sensors contributed evidence, how behaviour compares to expected patterns, what risk level is warranted given missing information, and what *authorised categories of response* a human may select. The platform that supports those needs is a software-intensive system of systems: sensor adapters, normalisation and quality services, durable event streaming, detection and tracking, multi-sensor fusion, behaviour and risk modules, operator consoles, identity and access control, audit, observability, and gated external integration.

The systems engineering problem is to define, interface, integrate, verify, and evolve that platform so that it remains correct under messy data, operable under cognitive load, fail-visible under partial outage, and safe under malicious or accidental misuse—over a multi-year ownership horizon.

1.2 Why systems engineering (not only software delivery)

Software delivery practices (CI/CD, microservices packaging, container orchestration) are necessary but insufficient. Counter-swarm decision support exhibits classic systems properties:

- **Heterogeneous interfaces.** Vendor radars, EO/IR cameras, RF sensors, acoustic arrays, and telemetry sources do not share schemas. Integration without a canonical observation contract produces brittle cores.
- **Emergent behaviour.** Track over-merge, alert storms, and operator over-trust of model scores emerge from component interactions, not from any single unit test.
- **Safety constraints that must survive software change.** “No autonomous destructive actuation” is a system property. It must be enforced in architecture, ICD, tests, deployment configuration, and operational procedure—not only in a README.
- **Staged exposure to physical reality.** Live sensors and field networks introduce power, thermal, RF, and authorisation constraints that simulation cannot fully replace, but that simulation *must* precede.

These properties align with the systems engineering lifecycle perspective in ISO/IEC/IEEE 15288:2015 (and successor editions): stakeholder needs, system requirements, architecture, integration, verification, validation, transition, and operation are distinct technical processes with explicit work products (ISO/IEC/IEEE, 2015). The INCOSE *Systems Engineering Handbook* elaborates how those processes interact with enabling processes such as risk management, configuration management, and quality assurance (INCOSE, 2015; INCOSE, 2023).

1.3 Ambiguities and underspecification

Any honest Stage 0 programme surfaces ambiguities. Material ones for this domain include:

1. Site CONOPS: single facility versus multi-site federation; air-gapped versus controlled egress.
2. Sensor vendor mix and whether sensor-native detections are authoritative or advisory.
3. Exact latency SLOs under operational clutter versus lab targets.
4. Legal and export-control classification of models and software artefacts.
5. Authority model for external category handoff (who may enable production integration endpoints).
6. Edge power/thermal envelopes and whether vision inference must run on-post.

Where assumptions exceed facts, this monograph marks them as assumptions and designs for reversibility (adapter boundaries, schema versioning, deploy profiles) rather than pretending certainty.

1.4 Assumptions (explicit)

ID	Assumption	If wrong
A-01	Development begins with simulation; live sensors arrive under authorisation.	Compress Stage 7 earlier; do not skip twin contract tests.
A-02	MVP is single security domain / single site.	Multi-site tenancy becomes a Stage 8+ programme with federation ICD.
A-03	External systems accept abstract authorised categories, not effector primitives.	Integration API remains category-only; effector systems stay outside platform.
A-04	Clocks can be synchronised within defined bounds; residual skew is quality-flagged.	Association gates widen; fusion confidence ceilings apply.
A-05	Operators are trained; English-first console initially.	Localisation and HF evaluation expand Stage 5.
A-06	Central-primary hybrid is acceptable for MVP reliability.	Edge promotion is packaging, not schema rewrite.

1.5 What must be true for success

1. A stable canonical observation schema and ICD that all adapters honour.
2. Human approval as a hard gate before any external handoff.
3. Tamper-evident audit for material events.
4. Simulation that exercises the same contracts as live adapters.
5. Observable degradation modes that operators can understand.
6. Traceability from requirements to architecture to tests.
7. A roadmap that forbids “coding past the safety boundary” under schedule pressure.

1.6 Scope honesty and defensive boundary

This monograph addresses **defensive decision support**: sensing, fusion, tracking, risk, human approval, audit, and category-only external notification. It does **not** provide weapon guidance, kinetic fire-control loops, electronic-attack execution parameters, or methods to bypass safety interlocks. Mock effectors and external simulators exist only to validate handoff reliability and failure handling.

A full research dissertation typically comprises 60,000–100,000 words of original empirical research. This document is a research-structured monograph: literature-backed design, failure analysis, V&V frameworks, and integration guidance with verified landmark citations. Depth expands in subsequent passes without inventing sources.

1.7 Structure of the monograph

Sections 2–4 establish standards context, mission, and MoSCoW. Sections 5–7 develop architecture, ICD, and hybrid compute. Sections 8–10 develop twin-first integration, FMEA, and proof. Sections 11–12 operationalise the Stage 0–8 roadmap and a practical build/integration guide. Section 13 closes with a pre-ship ownership gate.

2. Related standards and literature

2.1 Systems engineering process standards

ISO/IEC/IEEE 15288 (*Systems and software engineering — System life cycle processes*) defines a common process framework for man-made systems, covering agreement, organisational project-enabling, technical management, and technical processes including stakeholder needs and requirements definition, system requirements definition, architecture definition, design definition, system analysis, implementation, integration, verification, transition, validation, operation, maintenance, and disposal (ISO/IEC/IEEE, 2015). Citing 15288 carefully means using it as a *process reference*—not claiming formal certification. For counter-swarm platforms, the most leveraged processes are:

- Stakeholder needs and requirements definition (mission, users, CONOPS).
- Architecture definition (edge/central, safety boundaries).
- Integration and verification (ICD, twin, HIL).
- Validation (fitness for intended use with operators).
- Operation and maintenance (runbooks, SLOs, rollback).

ISO/IEC/IEEE 15289 addresses content of life-cycle information items and is useful when programmes must name the artefacts they will maintain (requirements specification, ICD, V&V plan, etc.) (ISO/IEC/IEEE, 2019).

IEEE 1220 historically addressed application and management of the systems engineering process; modern programmes more often map to 15288 and organisational handbooks, but the emphasis on requirements baseline and configuration control remains relevant (IEEE, 2005).

2.2 Handbooks and body of knowledge

The **INCOSE Systems Engineering Handbook** is the primary practical companion to 15288 for many defence and aerospace programmes. It elaborates technical and management processes, discusses system architecture concepts, and stresses verification and validation distinctions (INCOSE, 2015; fourth edition updates and subsequent editions, including INCOSE, 2023). For this monograph, INCOSE’s framing of:

- needs versus requirements,
- architecture as decision record,
- integration as progressive assembly against interfaces,
- verification as “built right” versus validation as “right system,”

is adopted directly.

The **NASA Systems Engineering Handbook** (NASA/SP-2016-6105 Rev2 and related guidance) is widely used for its clarity on requirements writing, technical performance measures, and integration/test philosophy in complex programmes (NASA, 2016). Its insistence that requirements be verifiable and that interfaces be controlled before detailed design is especially applicable to multi-sensor C2.

The **Defense Acquisition University / US DoD systems engineering guidance** (various SE guides and the Systems Engineering Body of Knowledge, SEBoK) provide additional vocabulary for operational requirements documents, interface control, and test and evaluation—useful even when a

programme is not a formal US DoD acquisition (SEBoK Editorial Board, ongoing; DAU SE resources).

2.3 Interface control and configuration

Interface control is older than microservices. Aerospace and defence programmes have long used Interface Control Documents to freeze mechanical, electrical, and data interfaces between configuration items. Software-intensive systems inherit the same discipline: **no implementation against undocumented interfaces**. Schema registries, OpenAPI specifications, and event contract tests are modern ICD mechanisms. Configuration management practices from ISO 10007 and organisational CM plans prevent “silent” interface drift (ISO, 2017).

2.4 Failure engineering and safety

IEC 60812 specifies procedures for failure mode and effects analysis (FMEA) and failure mode, effects and criticality analysis (FMECA) (IEC, 2018). The severity–occurrence–detection style risk priority number (RPN) used in many programmes is an *indicative* ranking aid, not a physical law; this monograph treats RPN as a prioritisation heuristic and insists on qualitative fail-safe principles alongside numeric scores.

MIL-STD-882E (*System Safety*) provides a DoD standard practice for system safety, including severity categories and risk acceptance—valuable for thinking about mishap potential even when the platform itself does not fire weapons (US DoD, 2012). For decision-support platforms, the safety case focuses on: incorrect operational picture leading to inappropriate human decisions; silent failure of audit; and accidental enablement of external integrations.

Functional safety standards such as **IEC 61508** (electrical/electronic/programmable electronic safety-related systems) are not a direct fit for all IT-centric C2 stacks, but their concepts of systematic capability, diagnostic coverage, and independence of safety functions inform software interlock design (IEC, 2010). Programmes should not claim SIL ratings without a competent safety assessment.

2.5 Digital twins and modelling and simulation

Digital twin literature spans manufacturing, aerospace, and cyber-physical systems. Grieves’ early framing of the digital twin as a virtual representation linked to a physical product, and subsequent NASA/US Air Force discussions of digital twins for vehicles, establish the conceptual lineage (Grieves, 2014; Glaessgen & Stargel, 2012). For counter-swarm *software* platforms, the operative twin is less a photoreal 3D world and more a **controlled world model that emits synthetic observations through production adapter contracts**, with a scorer comparing platform estimates to withheld truth. Modelling and simulation best practice from military M&S communities emphasises verification of models, validation against intended use, and accreditation (VV&A) where required (Department of Defense M&S guidance; Sargent, 2013 on simulation validation).

2.6 Multi-sensor fusion and tracking (systems view)

Landmark tracking and fusion literature—Kalman filtering, multiple hypothesis tracking, probabilistic data association, and information fusion architectures (Bar-Shalom et al., 2001; Hall & Llinas, 1997; Chong et al. on distributed fusion)—matters here as *capability context*, not as algorithm prescriptions. Systems engineers need enough of this literature to specify interfaces (state, covariance, evidence links, coasting behaviour) and evaluation metrics (MOTA/IDF1-class measures, association error), while leaving algorithm selection to ML/tracking specialists under ICD constraints.

2.7 Human factors and decision support

Endsley’s situation awareness model and related human factors work on displays under uncertainty remain relevant when defining operator console requirements (Endsley, 1995). Decision-support systems must distinguish observation, inference, prediction, recommendation, and decision—epistemic hygiene that product design and systems engineering jointly own.

2.8 Software architecture references

Bass, Clements, and Kazman’s *Software Architecture in Practice* and related SEI work on quality attribute scenarios provide a bridge from systems quality attributes (latency, availability, safety, security, modifiability) to architectural tactics (Bass et al., 2012). Event-driven architectures for sensor systems follow established patterns for durable logs and replay (Kreps et al. on Kafka’s design lineage; modern Kafka-API compatible brokers).

2.9 Positioning of this monograph

Relative to the above, this work does not invent a new lifecycle standard. It specialises existing SE practice to a concrete defensive platform class with: (a) a hard safety boundary at category-only handoff; (b) hybrid edge hardware rules that prevent Raspberry Pi from being mistaken for an operational fusion brain; (c) twin-before-HIL as a programme gate; and (d) FMEA linked to chaos tests and degradation modes.

3. Mission definition and operational concept

3.1 Operational problem

Defenders must form a timely, evidence-backed air picture from heterogeneous sensors and support human decisions about authorised response categories under uncertainty—including possible coordinated or swarm-like behaviour—without autonomous harmful effects.

3.2 System objectives

1. Reliable multi-sensor ingest and normalisation.
2. Coherent tracks with provenance (evidence observation IDs).
3. Behaviour and risk decision support with explicit uncertainty and missing-information flags.
4. Operator-usable console under alert load.
5. Governed AI/ML with versioned inference and audit.
6. Controlled external integration (category handoff only).
7. Testability via digital twin before hardware exposure.

3.3 Users and stakeholders

Stakeholder	Primary need
Security operator	Air picture, alerts, decide/dismiss/escalate
Sensor operator	Health, cueing recommendations (non-effector)
Incident commander	Priority, authorisation, escalation path
Analyst	Replay, after-action, evaluation packages
System administrator	Deploy, identity, configuration
Governance / security officer	Audit integrity, model governance, field gates
Integrator	Adapter compliance, ICD fixtures

3.4 Operating environments (assumed)

- Secured facility or field network.
- Mixed edge sensor posts.
- Central operations node.
- Simulation / digital-twin laboratory.

Exact RF environments, weather libraries, and classified threat libraries are site-specific and are not required inside a public research repository.

3.5 Quality attribute objectives (draft)

Attribute	Draft objective
Latency	Observation→normalised event p95 ≤ 200 ms (in-cluster sim path); track visible p95 ≤ 1 s; console paint p95 ≤ 1.5 s
Availability	99.5% single-site MVP for control plane + core pipeline
Safety	No autonomous destructive actuation; fail closed on audit/IAM for decisions
Security	Zero-trust oriented service identity + RBAC; TLS; secrets outside VCS
Explainability	Evidence-linked assessments; model version on critical outputs
Modifiability	Vendor independence via adapters; schema BACKWARD compatibility within major

These are design targets pending site CONOPS. Systems engineering owns the negotiation of targets against cost and risk; it does not invent false precision.

3.6 Mission threads (operational scenarios)

Mission threads force architecture and ICD early:

1. **Single benign object** — bird-like kinematics; expect low risk, no alert storm.
2. **Single UAV-like track across overlapping sensors** — association and fusion evidence.
3. **Crossing tracks** — association stress; prefer under-merge to silent over-merge in ambiguous cases when policy so states.
4. **Coordinated multi-object patterns** — behaviour indicators with uncertainty; never sole authorisation basis.
5. **High clutter** — false observations; operator cognitive load.

6. **Missing modality** — RF silent; risk lists missing information.
7. **Contradictory class evidence** — conflict flags retained.
8. **Clock skew / delayed observations** — quality flags; widened gates.
9. **Sensor death mid-run** — coverage map; no synthetic tracks.
10. **Network partition and heal** — edge buffer; resync; stale-picture banner.
11. **Alert storm** — suppression and aggregation.
12. **Human decision + handoff success/fail** — audit continuity; DLQ behaviour.

3.7 Non-missions (explicit)

The platform does not: aim weapons; compute fire-control solutions; emit jamming waveforms; select human targets; or silently escalate from recommendation to command. External authorised systems—if any—are separately engineered, accredited, and connected only through the category handoff API.

4. Requirements engineering and MoSCoW

4.1 Requirements as engineered artefacts

Per NASA and INCOSE guidance, a requirement should be necessary, implementation-independent where possible, clear, verifiable, and traceable (NASA, 2016; INCOSE, 2015). For this platform, requirements IDs (FR-, *NFR*-) form a controlled vocabulary mapped to architecture components, ICD entries, and tests.

4.2 MoSCoW prioritisation

MoSCoW (Must, Should, Could, Won't) is a prioritisation technique widely used in agile and product contexts; used carefully in systems programmes it prevents schedule pressure from expanding scope into safety-critical forbidden territory.

Must have

- Canonical observation schema + pluggable adapters.
- Event bus with validation and quarantine.
- Tracking + multi-sensor association (baseline algorithms acceptable).
- Risk/priority with uncertainty and missing-info listing.
- Operator console: map, tracks, alerts, evidence, timeline, health.
- Human decision capture with RBAC.
- Tamper-evident audit for material events.
- Simulation environment consuming production contracts.
- Observability: health, latency, lag, errors.
- Safety: no autonomous destructive effects; category-only external API.

Should have

- Behaviour / coordination indicators.
- Model registry + versioned inference.
- Tool-bounded AI assistant for summarisation / NL query.
- Replay and after-action packages.

- Schema registry and contract tests in CI.
- Edge/central split design (edge optional early).

Could have

- Multi-site tenancy.
- Advanced Bayesian intent models.
- Native hardened clients.
- Automated dual-control workflows.
- Rich 3D twin visualisation.

Won't have (this programme)

- Weapon control loops.
- Autonomous jamming / kinetic tasking.
- Open public multi-tenant SaaS without a separate hardening programme.
- Pi-as-fusion-brain architectures.

4.3 Functional requirement clusters (summary)

Sensor ingestion. Pluggable adapters; vendor types stop at adapter boundary; simulated sensors indistinguishable at schema boundary; tolerate silence.

Normalisation and quality. Common CRS + UTC; quarantine with reason codes; idempotency; versioned schema evolution.

Event streaming. Durable publish; idempotent consumers; lag metrics; replay from retention.

Detection / classification / tracking. Hypotheses with scores; track IDs and state; coast/reacquire observable; evaluation under clutter in twin.

Fusion. Cross-sensor association; contradictions retained; evidence citation.

Behaviour and risk. Indicators with uncertainty; behaviour never sole authorisation; risk graded with rationale; rules/models versioned.

Alerts and decisions. Policy-based alerts; recommendations are categories only; humans decide; mandatory audit fields.

External integration. Versioned APIs; category + context; explicit enablement per environment; mock external in sim.

IAM and ML governance. Authenticated actions; least privilege; model cards (version, owner, intended use, limitations); LLM tools cannot bypass policy.

4.4 Non-functional requirement clusters (summary)

Latency, availability, scalability, security, maintainability/testability, observability, recoverability, and explainability NFRs are design targets. Example lab ingress target: $\geq 2k$ observations/s sustained with headroom plan to 10k; ≥ 500 concurrent tracks in MVP lab profile. Draft RPO ≤ 1 min and RTO ≤ 30 min for hot store MVP are planning numbers pending CONOPS.

4.5 Traceability discipline

Each FR/NFR maps to: architecture component, ICD ID(s), and test layer (unit, contract, sim, chaos, security, HF). Traceability is not paperwork theatre; it is how programmes prove that “Must” items were not silently dropped when demos looked good.

4.6 What goes wrong in requirements

Failure	Symptom	Mitigation
Solution-posing requirements	“Must use neural net X”	Restate as capability + performance + constraints
Unverifiable NFRs	“Real-time” without percentiles	Bind to p95/p99 and environment
Scope creep into effectors	“Just one fire API”	Won’t-have enforcement in ICD + safety tests
Assumption buried as fact	Site latency treated as known	Open questions register; reversible design
Orphan Musts	No test owns the FR	Traceability gate on Stage exit

5. Architectural approaches and justified selection

5.1 Architectural drivers

1. Heterogeneous, late, missing, contradictory data is normal.
2. Operator cognitive load is a first-class constraint.
3. Safety boundary must be enforceable in software and process.
4. Vendor independence via adapters.
5. Simulation-first delivery.
6. Prefer boring, proven infrastructure unless justified.

5.2 Approach A — Monolith “C2 application”

Core idea. One deployable application owns ingest, tracking, UI, and handoff.

Gains. Simple deployment; easy local reasoning early.

Losses. Sensor rate couples to UI; hard to isolate ML lifecycle; weak independent scale; safety boundaries blur inside one process space.

When it becomes a liability. As soon as multiple sensor rates and model versions coexist.

Verdict. Reject as production target. Optional modular monolith only as a time-boxed spike, not as the architecture of record.

5.3 Approach B — Fine-grained microservices everywhere

Core idea. Maximal service decomposition; every function a separately deployed unit.

Gains. Independent scale; clear team ownership in large orgs.

Losses. Operational burden, distributed debugging, premature network partitions between tightly coupled track/fusion steps.

When it becomes a liability. Small teams before Stage 4; on-call explosion.

Verdict. Reject extreme split. Prefer modular services along domain boundaries.

5.4 Approach C — Event-driven modular services (selected)

Core idea. Durable event stream as the integration backbone; modular services for adapters, normalisation, detection, tracking, fusion, behaviour, risk, alert, approval, audit, operator API, realtime gateway, inference gateway, integration API, sim engine.

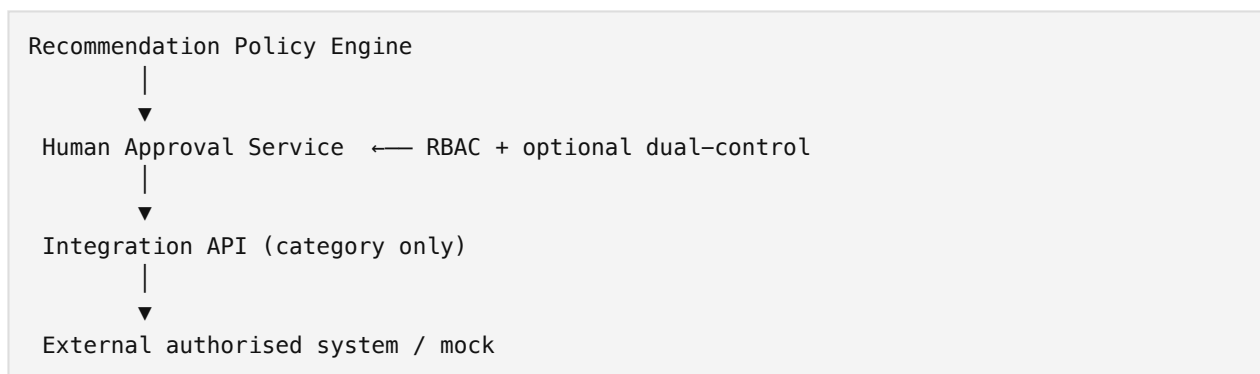
Gains. Fits asynchronous sensors; replay for recovery and tests; decoupling; natural twin injection point.

Losses. Eventual consistency complexity; need explicit consistency notes in ICD; requires bus operations skill.

When it becomes a liability. If the team substitutes “DB as queue” under skill pressure—usually regenerating the same problems without the tooling.

Verdict. Selected. Budget training/ops time rather than deleting the bus.

5.5 Safety architecture (cross-cutting)



Software interlocks:

- No service credentials in this platform’s integration layer that invoke weapon interfaces.
- Recommendation ≠ command; separate event types.
- Demo mode cannot enable real integration endpoints without governance-approved configuration.
- Audit write failure fails closed for decisions that require audit.

5.6 Data architecture summary

- **Hot:** PostgreSQL + PostGIS for tracks, incidents, config.
- **Stream:** Kafka-compatible bus (Redpanda candidate for lab ops simplicity).
- **Ephemeral:** Redis for sessions/rate limits—not system of record.
- **Cold:** S3-compatible object storage for raw media and model artefacts.
- **Analytics:** Optional warehouse Stage 4+; start with Postgres + export.

The canonical observation schema is the stability hinge of the entire programme.

5.7 Architecture decision records (initial)

ADR	Decision	Status
ADR-001	Event-driven modular services	Proposed
ADR-002	Postgres + PostGIS hot store	Proposed
ADR-003	Kafka-API bus (Redpanda candidate)	Proposed
ADR-004	Web operator console first	Proposed
ADR-005	Central-primary hybrid edge	Proposed
ADR-006	Category-only external API	Accepted (safety)
ADR-007	Simulation-first delivery	Accepted

5.8 What should not be built yet (or ever in-platform)

Item	Why
Effector fire-control	Safety / scope
LLM deciding risk alone	Reliability / governance
Per-vendor core schemas	Lock-in
Multi-region active-active MVP	Complexity vs value
Custom DB engine	Unjustified
Novel blockchain for audit	Prefer hash-chained append log or immutable storage

6. Interface control documentation

6.1 ICD as the primary integration artefact

An Interface Control Document states, for each interface: producer, consumer, protocol, format, authentication, latency budget, retry semantics, failure behaviour, versioning policy, and observability signals. The governing rule: **no implementation against undocumented interfaces**.

Modern ICD content for this platform is a mix of:

- Event schemas (`observation.v1`, `detection.v1`, `track.update.v1`, ...).
- OpenAPI for operator and integration HTTP APIs.
- gRPC contracts for high-rate adapter→normaliser and inference.
- Controlled vocabularies for response categories and alert tiers.

6.2 Interface index

ID	Interface
ICD-01	Sensor/Adapter → Normaliser
ICD-02	Normaliser → Event Bus (<code>observation.v1</code>)
ICD-03	Event Bus → Detection / Tracking / Quality
ICD-04	Detection → Event Bus (<code>detection.v1</code>)
ICD-05	Tracking/Fusion → Event Bus (<code>track.update.v1</code>)
ICD-06	Behaviour → Event Bus (<code>behaviour.indicator.v1</code>)
ICD-07	Risk → Event Bus (<code>risk.assessment.v1</code>)
ICD-08	Alert → Event Bus / Operator API (<code>alert.v1</code>)
ICD-09	Operator UI ↔ Operator API / Realtime
ICD-10	Approval → Audit + Decision store
ICD-11	Integration API → External system
ICD-12	Inference Gateway ↔ Models
ICD-13	AI Assistant ↔ Tool Policy ↔ Operator API
ICD-14	Services → Observability backend
ICD-15	Sim Engine → Sim Adapters

6.3 Conventions

- Versioning: additive changes in minor; breaking in major. Consumers ignore unknown fields.
- Schema registry compatibility: BACKWARD within a subject where feasible.
- Idempotency keys on produce and on human decisions.
- Quarantine topics for validation failures—never crash the bus on poison input.
- Every ICD entry requires a contract test fixture before the producing service merges a schema change.

6.4 Critical interface notes

ICD-01 / ICD-15. Real and simulated adapters must be indistinguishable at the schema boundary. This is the twin-first hinge.

ICD-05. Track updates carry state, covariance or quality, evidence observation IDs, and coasting markers. Duplicate handling is mandatory.

ICD-10. Human decisions use user JWT + `decision.write`; interactive persist $p95 \leq 300$ ms; no silent success on failure.

ICD-11. Payload shape: { `decision_id`, `category`, `site_id`, `track_ids`, `evidence_digest`, `timestamp` }. **Forbidden fields:** weapon aimpoints, RF jam parameters, fuse arming, and similar effector primitives. Failure leaves local decision valid with handoff status failed.

ICD-13. Assistant tools are allowlisted; core console remains usable if assistant fails; tool calls are audited.

6.5 Controlled vocabularies (examples)

Response categories (final list customer-owned): DISMISS, HEIGHTEN_MONITORING, CUE_SENSOR, NOTIFY_EXTERNAL, REQUEST_ESCALATION, OPEN_INCIDENT. None implies kinetic or EA execution.

Alert tiers: T0 informational, T1 advisory, T2 operator action suggested, T3 commander attention.

6.6 What goes wrong without ICD discipline

Failure	Effect	Detection	Fix
Vendor types leak into core	Lock-in; dual schemas	Code search / review gates	Adapter boundary tests
Breaking schema without major	Consumer poison loops	Compatibility CI	Registry + contract tests
Recommendation event reused as command	Safety bypass	Safety test suite	Separate types + deny paths
UI polls DB that tracking also writes	Hidden coupling	Architecture review	Projections via bus
“Temporary” undocumented field	Becomes permanent API	Schema diff in PR	ICD change request process

6.7 ICD change control

Treat ICD changes as configuration items: propose, impact analysis (consumers, twin fixtures, HIL adapters), dual-run if needed, deprecate, remove. Systems engineering owns the register; software engineering owns implementation; security/governance co-owns ICD-11 and any autonomy-adjacent change.

7. Hybrid edge–central compute integration

7.1 The placement question

Where does compute run, and should edge nodes be Raspberry Pi or NVIDIA Jetson-class hardware? This is a systems decision driven by bandwidth, latency, thermal/power, fleet operations, and credibility under load—not by hobby familiarity.

7.2 Tier model

Tier	Typical hardware	Role
Central / ops room	x86 server or workstation ($\pm$ discrete GPU)	Fusion, risk, operator console, bus, DB, audit, multi-sensor correlation
Edge — vision / heavy detect	NVIDIA Jetson (Orin Nano / Orin NX / AGX) or rugged GPU box	EO/IR inference close to camera; bandwidth reduction
Edge — light ingest	Industrial SBC; Raspberry Pi / CM4 where justified	Protocol adapters, health, buffering — not primary ML
Radar / RF front-end	Vendor appliance or FPGA/SDR host	Often opaque; platform talks via adapter only

7.3 Comparison

Criterion	Raspberry Pi 5 / CM4	NVIDIA Jetson Orin class	Central x86 + GPU
Cost / availability	Low / easy	Medium	Higher
EO detection (YOLO-class)	Marginal / constrained	Strong	Strong
Multi-sensor fusion site-wide	Weak	Local only	Strong
Ops maturity for AI	Hobby/lab leaning	Industrial edge AI norm	Datacentre/ops norm
Power / rugged options	Limited unless carrier board	Better ecosystem	Best in ops room
Maintenance	Simple OS	NVIDIA BSPs / JetPack	Standard Linux/K8s
Fit	Lab adapters, light gateways	Vision edge posts	Primary platform

7.4 Selected baseline

Central x86 + optional NVIDIA edge + Pi only where justified.

Rules:

1. **Stage 1:** everything on one lab x86 host (Docker Compose) + software simulator. No Pi/Jetson required to start.
2. **Vision edge (Stage 2–3+):** prefer Jetson Orin class when real cameras arrive.
3. **Raspberry Pi:** lab teaching, cheap serial/GPIO/acoustic prototypes, or light protocol gateways —not the fusion brain, not the primary EO detector in a serious field trial.
4. **Do not** pin the core platform to Pi hardware APIs.

7.5 Function placement matrix

Function	Edge	Central
Adapter + light preprocess	Yes	Optional
Hard real-time EO detector	Prefer when bandwidth-bound	Optional
Multi-sensor fusion across posts	Limited local	Yes
Behaviour/risk across site	No	Yes
Operator console	Thin/cache	Yes
Audit system of record	Buffer + forward	Yes
Model training	No	Yes

7.6 Trade-offs

Choice	Gains	Losses / risks
Central-first	Faster delivery; one ops story	Needs network to edges for live sensors
Jetson at EO edge	Lower video bandwidth; lower detect latency	JetPack/version drift; fleet management
Pi as main edge AI	Cheap demos	Thermal/CPU limits; credibility gap
Everything on Jetson cluster	Homogeneous story	Painful to run Postgres/Kafka-class services

7.7 Stage mapping for hardware

Stage	Compute
0	Docs only
1	Single x86 lab machine + Compose
2–4	Same + richer sim; optional first Jetson for camera adapter experiments
5–6	Central hardened; edge profiles documented
7 HIL	Representative Jetson + vendor/test sensors
8 Field	Site-approved hardware list (may add rugged PCs)

7.8 Edge integration pattern

PHYSICAL SENSOR → VENDOR SDK → ADAPTER → CANONICAL OBSERVATION → PLATFORM
PLATFORM → CONTROLLED API → EXTERNAL SYSTEM (category only)

Edge buffers are bounded; shed oldest with metrics; never assume infinite local disk. On partition, central shows stale-picture degradation; on heal, resync from watermarks without inventing tracks.

7.9 Open stakeholder decisions that affect hardware

- Air-gapped or connected site?
- Camera/radar vendors?
- Power/thermal envelope at edge posts?

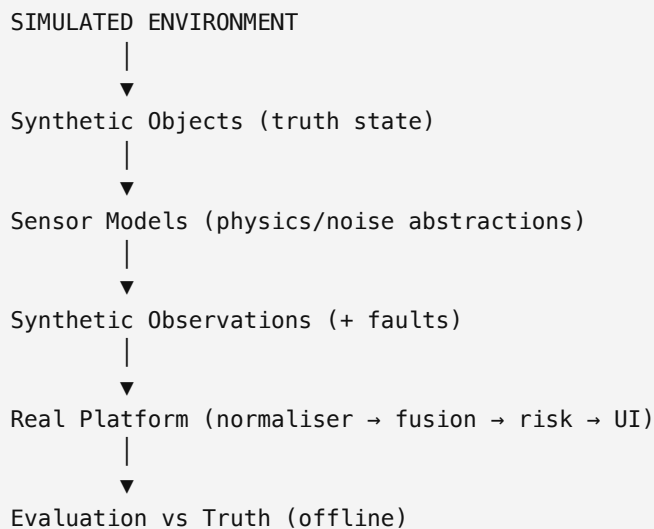
- Budget ceiling per edge node?

Until answered, engineering proceeds on Compose-on-x86 and designs adapters so Jetson packaging is a **deploy profile**, not a rewrite.

8. Digital twin before hardware-in-the-loop

8.1 Purpose

The digital twin provides a controlled world model that emits synthetic sensor observations (and faults) into the **same adapter contracts** as real sensors. Its purpose is to develop and validate the full software platform without operational hardware or harmful effects.



8.2 Why twin before HIL

HIL introduces vendor SDKs, timing quirks, power, cabling, and facility constraints. If the platform's contracts, idempotency, quarantine, coasting, and operator degradation modes are not already proven in twin, HIL becomes an expensive debugger for software mistakes. Twin-first is therefore a **cost and safety control**, not a research ornament.

This stance aligns with modelling and simulation VV&A thinking: know what the model is for, verify it is built correctly for that purpose, validate it is suitable for intended use, and do not over-claim (Sargent, 2013; DoD M&S VV&A practice).

8.3 Twin components

Component	Role
Scenario director	Loads scenarios; time control; seed
Truth engine	Object kinematics, swarm scripts, environment flags
Sensor models	Radar/EO/RF/acoustic abstractions: FOV, rate, noise, bias
Fault injector	Drop, delay, duplicate, corrupt, contradict, sensor death, link loss
Sim adapters	Publish candidate observations like real adapters
Mock external system	Receives category handoffs; canned acks/failures
Scorer	Compares tracks/risk to truth for metrics
Recorder	Saves scenario run bundles for regression

8.4 Must-have scenario suite

1. Single non-threatening object (bird-like).
2. Single UAV-like track across overlapping sensors.
3. Crossing tracks (association stress).
4. Coordinated multi-object swarm-like patterns.
5. High clutter / false observations.
6. Missing modality (RF silent).
7. Contradictory class evidence.
8. Delayed observations / clock skew.
9. Sensor failure mid-run.
10. Network partition and heal.
11. Alert storm potential (tuning).
12. Operator decision + handoff success/fail.

8.5 Truth versus estimate (human factors constraint)

Truth stays inside the twin and scorer. It must not be shown as live fact to operators during human-factors trials in a way that trains over-trust. An instructor view, if any, is separate and role-gated.

8.6 Non-goals of the twin

- Photoreal battlefield rendering as a hard dependency.
- Classified threat libraries in a public repo.
- Real RF transmission.

Abstract kinematics plus statistical sensor models are sufficient for architecture proof and many fusion/ML evaluations.

8.7 Twin role by stage

Stage	Twin role
1	Prove bus→UI path
3–4	ML and fusion metrics
5	UX workload studies
6–7	Chaos + HIL bridge (hybrid truth)
8	Regression oracle for authorised field findings

8.8 What goes wrong with twin programmes

Failure	Effect	Mitigation
Twin bypasses adapters	False confidence; HIL shock	ICD-01/15 identity
Truth leaked to operators	Over-trust	Role gating
Only sunny-day scenarios	Untested degradation	Fault injector mandatory
Unversioned scenarios	Irreproducible claims	Seed + scenario IDs in recorder
Scorer metrics gamed	Inflated Stage exits	Pre-registered thresholds; slice metrics

9. Failure engineering and FMEA

9.1 Guiding fail-safe principles

1. Prefer **fail visible** over fail silent.
2. Prefer **degrade coverage** over inventing tracks.
3. Prefer **block external handoff** over uncertain automation.
4. Prefer **preserve audit** even if UI is impaired.
5. Recommendations never become commands on failure.

These principles implement the spirit of system safety practice (MIL-STD-882E thinking about mishap severity) without claiming a full DoD safety programme.

9.2 FMEA method

Per IEC 60812-style analysis: identify component failure modes, local and system effects, detection means, mitigations, and residual risk. Severity (1–5), Likelihood (1–5), Detectability (1–5; higher = harder to detect), $RPN = Sev \times Lik \times Det$ as an indicative rank only.

9.3 Selected high-value FMEA rows

ID	Component	Failure mode	Effect	Sev	Lik	Det	RPN	Detection	Mitigation / recovery
FM-01	Sensor	Hard silence	Coverage hole	4	4	2	32	Heartbeat	Coverage map; confidence ceiling; T1 alert
FM-02	Sensor	Bias/miscal	Systematic track error	4	3	4	48	Cross-sensor residual	Quarantine sensor; calibration workflow
FM-03	Adapter	Crash loop	Loss of modality	3	3	2	18	Restart metrics	Supervisor restart; buffer
FM-04	Network edge↔central	Partition	Delayed/lost picture	5	3	2	30	Link probes	Edge buffer; degraded UI; resync
FM-05	Event bus	Broker outage	Pipeline stop	5	2	1	10	Bus health	Multi-broker; backpressure; banner
FM-06	Event bus	Poison message	Consumer stuck	4	2	3	24	DLQ / error budget	Skip-to-DLQ; schema guards
FM-07	Normaliser	Bad CRS map	Geospatial wrong	5	2	4	40	Geo unit tests	Versioned transforms; kill-switch
FM-08	Clock sync	Skew	Mis-association	4	3	3	36	Skew metrics	Quality flag; widen gates; NTP/PTP monitor
FM-09	DB hot	Primary down	API/query fail	4	2	1	8	Ready checks	Failover; freeze decisions if needed
FM-10	Tracking	Divergent state	Duplicate/split tracks	3	3	3	27	Track KPIs	Replay reinit; tuning
FM-11	Fusion	Over-merge	Two objects as one	4	3	4	48	Operator reports; sim	Conservative assoc; evidence UI
FM-12	Fusion	Under-merge	Fragmentation	3	3	3	27	Track count vs truth	Tune; often safer than over-merge
FM-13	Model inference	Timeout/down	Missed detections	4	3	2	24	Infer errors	Passthrough/last-known policy; rollback
FM-14	Model inference	Silent bad outputs	Wrong class	5	2	5	50	Calibration monitors	Canary; disable route; evidence norms
FM-15	Risk engine	Rule bug	Mis-prioritisation	4	2	3	24	Policy tests	Version pin; feature flag
FM-16	Alert service	Storm	Operator overload	4	3	2	24	Alert rate SLI	Aggregate; suppress;

									breakers
FM-17	Operator console	Blank/crash	Loss of HCI	5	2	1	10	RUM/synthetic	Redundant workstation; backup channel
FM-18	Approval API	Down	Cannot decide	4	2	1	8	Health	Block handoff; ops paper procedure
FM-19	Integration API	Down/DLQ	External not notified	3	2	2	12	Handoff metrics	Retry/DLQ; local record remains
FM-20	Audit store	Write fail	Compliance break	5	2	2	20	Audit errors	Fail closed on decisions needing audit
FM-21	Object store	Outage	No raw replay	2	2	1	4	S3 errors	Hot path continues if metadata enough
FM-22	Corrupt message	Partial JSON	Consumer errors	3	3	2	18	Schema validation	Quarantine
FM-23	Duplicate burst	Reorders	Inflated counts	2	4	2	16	Idempotency	Dedup keys
FM-24	Contradictory obs	Conflict	Confused UI	3	4	2	24	Conflict flags	Show conflict; no silent auto-resolve
FM-25	AI assistant	Hallucination	Operator misled	3	4	3	36	Grounding checks	Citations; not on critical path
FM-26	Secrets leak	Credential theft	Broad compromise	5	2	4	40	Scanning; vault audit	Rotate; least privilege
FM-27	Edge compute	Hardware fail	Local loss	3	3	2	18	Device health	Spare; central continues

9.4 Graceful degradation modes

Mode	Trigger	Behaviour
D1 Coverage reduced	Sensor loss	Map hatching; no synthetic tracks
D2 Model degraded	Infer errors	Sensor-native detections only; badge
D3 Stale picture	Bus lag	Banner with lag age; dim tracks
D4 Decision freeze	Audit/IAM fail	View-only
D5 Integration offline	External down	Local decisions only
D6 Assistant off	AI policy trip	Hide assistant

9.5 Recovery and replay sequence

1. Restore bus/DB health.
2. Replay observations from retention watermark.
3. Rebuild projections.
4. Verify audit continuity.
5. Exit degraded mode with operator acknowledgement.

9.6 Redundancy recommendations

Component	MVP	Hardened
Bus	Multi-broker single AZ	Multi-AZ
DB	Primary + replica	Auto failover
Gateway	2 replicas	N+1
Edge buffer	Optional	Required for field

9.7 Living FMEA

FMEA is not a Stage 0 PDF to archive. Each chaos test maps to FM-IDs. Stage 7 adds hardware-specific rows. Site CONOPS recalibrates severity. Residual high-RPN items (FM-02, FM-11, FM-14 especially) demand ongoing detection investment because detectability is the weak factor.

10. Verification, validation, and proof

10.1 Verification versus validation

Following INCOSE and 15288 usage:

- **Verification** asks whether the system was built right: does it satisfy specified requirements and interface contracts?
- **Validation** asks whether the right system was built: does it satisfy stakeholder needs in intended use contexts?

Demos that “look good on a map” are neither. Proof requires planned evidence.

10.2 Test layers

Unit. Geo transforms, association gates, risk rules, authz helpers, schema validators.

Integration. Service pairs along ICD edges: adapter→normaliser, normaliser→bus, tracking←detections, approval→audit, handoff→mock external.

Contract. OpenAPI consumer/provider checks; event schema BACKWARD compatibility; ICD fixtures per interface ID.

ML evaluation. Offline metrics as applicable (precision, recall, F1, mAP, FPR/FNR, MOTA/IDF1-class, latency, calibration/ECE); slice metrics (range, clutter, lighting, swarm density); regression versus last promoted model; versioned gates for operator-critical routes.

Simulation. Scenario pack; golden runs in CI/nightly; scorer thresholds as release gates.

Chaos. Mapped to FMEA IDs: lost sensor, lost network, corrupt/delay/duplicate messages, DB unavailable, model failure, bus partition, clock skew.

Security. AuthN/AuthZ positive/negative; IDOR; injection; rate limits; privilege escalation attempts; dependency/container scans; prompt-injection corpus for AI tools; schema fuzz on ingest.

Human factors (non-CI). Scripted operator trials: time-to-evidence, error rates, subjective workload.

10.3 Safety-specific proofs

Test	Expected
Recommendation cannot call integration without decision	Deny
Integration schema rejects effector-like fields	Reject
Demo flag cannot target prod integration URL	Block
Audit write failure blocks decision commit	Fail closed

These tests are non-negotiable Stage exit criteria for any environment that could reach external systems.

10.4 Environments

Env	Purpose
Local Compose	Dev
CI	Unit/contract/sim smoke
Lab twin	Nightly full scenarios
Staging hardened	Security + perf
HIL	Stage 7
Field	Stage 8 authorised

10.5 Example exit criteria

- Contract tests green on main.
- Sim scenarios 1–12 pass agreed thresholds.
- Chaos drills for FM-05/09/13/20 documented.
- Critical CVEs addressed per security SLA.
- No open Sev-1 security findings.
- Safety-specific proofs pass.
- Traceability matrix shows every Must FR/NFR owned by a test.

10.6 Measures of effectiveness and suitability

Systems programmes distinguish MOEs (operational outcomes) from MOSs (suitability: reliability, maintainability, usability) and TPMs (technical performance measures) (NASA, 2016). Example mappings:

Class	Example
TPM	Bus consumer lag p95; observation accept latency
MOE	Time from first correlated multi-sensor evidence to operator-acknowledged alert
MOS	Mean time to restore pipeline after injected bus outage; operator workload scores

10.7 How to prove the system (evidence dossier)

A credible “we proved it” package includes:

1. Requirements baseline and MoSCoW with signed deltas.
2. Architecture + ADR pack.
3. ICD + schema registry history.
4. Twin scenario catalogue with seeds and thresholds.
5. Test reports: contract, sim, chaos, security, HF.
6. FMEA revision and residual risk acceptance.
7. Model cards and promotion records for any ML on critical path.
8. Runbooks: degrade modes, replay, rollback.
9. Limitations document (what the system does not claim).
10. Stage exit approvals including SG for field/HIL.

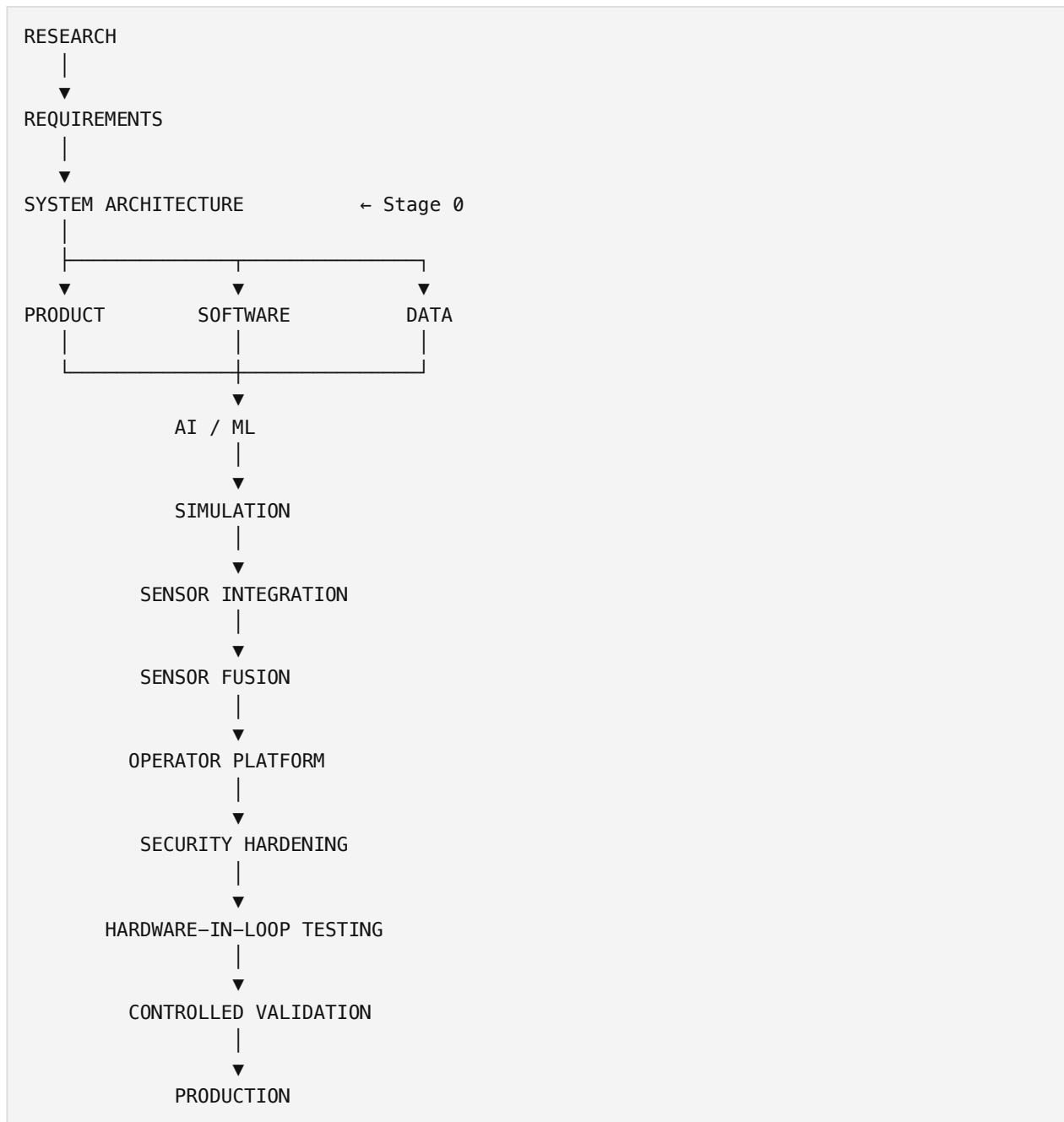
Without (9) and (10), demos tend to over-promise.

10.8 What goes wrong in V&V

Failure	Symptom	Fix
Testing implementation accidents	Brittle tests	Test against ICD
Metrics without slices	Hidden failure regimes	Slice evaluation
HF skipped	Pretty UI, poor decisions	Stage 5 HF gate
Security deferred to “later”	Field blockers	Stage 6 before Stage 8
Validation = customer watched a demo	No fitness evidence	Scripted trials + MOEs

11. Roadmap stages 0–8

11.1 Visual roadmap



11.2 Stage 0 — Research

Deliver: problem definition, user research synthesis, threat model, requirements, assumptions, technology research, architecture pack (system, ICD, FMEA, twin, roadmap, testing strategy, observability, open questions).

Exit: Document approval by stakeholders.

Code: None beyond scaffolding folders.

Gate: No Stage 1 coding until Stage 0 documents approved.

11.3 Stage 1 — Digital prototype

Sim → bus → fusion/tracking (baseline) → risk (rules) → dashboard skeleton.

Goal: Prove architecture wiring on a single x86 Compose host.

Exit examples: ICD-01/02/05/09 smoke; one sunny-day and one fault scenario; health endpoints live.

11.4 Stage 2 — Sensor integration patterns

Representative non-operational sources + richer sim sensors; validate schemas, timestamps, reliability; optional first Jetson camera adapter experiment without making Jetson the C2 host.

11.5 Stage 3 — ML pipeline

Detection/classification/tracking models as justified; registry; evaluation; rollback drills; inference gateway version pins.

11.6 Stage 4 — Multi-sensor fusion

Heterogeneous, contradictory, incomplete observation tests at scale in twin; association policy freeze candidates; evidence UI hooks.

11.7 Stage 5 — Operator platform

Map, tracks, confidence, evidence, alerts, timeline, health, audit UX polish; human-factors evaluation; assistant only after core UX stable.

11.8 Stage 6 — Security hardening

RBAC, service identity, encryption, secrets, scanning, monitoring, audit integrity verification; Compose→Kubernetes transition as justified; staging hardened environment.

11.9 Stage 7 — Hardware-in-the-loop

Representative hardware/test systems; still no real harmful effects; measure latency/reliability/recovery; extend FMEA with hardware rows; Jetson + vendor/test sensors as applicable.

11.10 Stage 8 — Controlled field testing

Only with authorisation; response systems abstracted/simulated unless separately engineered by qualified specialists; limitations signed; on-call and SLOs drafted for production transition.

11.11 Production transition

Reproducible deploy, runbooks, SLOs, on-call, limitation docs signed. Production is not Stage 8 by another name; it is a sustained operations commitment.

11.12 Parallelism after Stage 0 exit

Track	Start after
Sim engine + schemas	Stage 0 exit
Platform services skeleton	Stage 0 exit
Console IA implementation	Stage 0 exit (with product design)
Baseline tracker (classical)	Early Stage 1
ML detectors	Stage 3 (data ready)
AI assistant	After Stage 5 core UX stable
K8s hardening	Stage 6

11.13 Definition of done (programme)

Not “it runs.” Complete only when requirements, architecture, ICD, threat model, data model, simulation, tests, ML eval, failure tests, security controls, audit, monitoring, validated operator workflows, documented hardware boundaries, reproducible deploy, rollback, explainability, and limitations are satisfied.

11.14 Stage gate anti-patterns

Anti-pattern	Why it fails
Coding during unresolved Stage 0 contradictions	Rework multiplies at ICD layer
Skipping twin fault scenarios	HIL becomes software triage
Calling Stage 5 “done” without HF	Operator error dominates risk
Field trial without Stage 6	Security findings halt operations
Enabling real integration “just for the demo”	Safety boundary collapse

12. Build and integration guide

12.1 Integration stance

Do not concatenate specialist proposals uncritically. Challenge contradictions:

Tension	Resolution
Rich AI UX vs security risk	Assistant off critical path; epistemic UI rules
ML wants edge GPU vs ops cost	Central-primary MVP; edge-ready packaging
Data engineering wants many stores vs simplicity	Postgres + bus + S3 only for MVP
Fine microservices vs timeline	Modular services, not nanoservices

12.2 Recommended technology baseline (justified briefly)

Area	Selected	Explicit non-goal
Streaming	Redpanda (Kafka API) or Kafka	Custom broker
UI API	REST OpenAPI	GraphQL unless later justified
Internal infer	gRPC	Ad-hoc HTTP without schema
DB	Postgres + PostGIS	Multi-DB sprawl MVP
Cache	Redis	Redis as system of record
Objects	S3 API	Local disk as prod store
Backend	FastAPI/Python (+ Go if profiling demands)	Premature polyglot explosion
Frontend	React TypeScript SPA	Native MVP
Deploy lab	Docker Compose	K8s day-1 mandate
Deploy hardened	Kubernetes	Unreviewed PaaS lock-in
Auth	OIDC + mTLS services	Roll-your-own auth
Central compute	x86	Pi as C2 host
Vision edge	Jetson Orin class	Pi as primary detector
Light edge	Pi optional	Pi-only architecture

Novelty is not a reason to select technology (Bass et al., 2012 quality-attribute discipline; programme rule: prefer boring proven infrastructure).

12.3 Progressive integration sequence

1. **Freeze canonical observation fields** (even as draft) and ICD-01/02 fixtures.
2. **Stand up bus + schema validation + quarantine** in Compose.
3. **Sim adapter** → **normaliser** → **bus** → **consumer that logs**.
4. **Baseline tracker** producing `track.update.v1`.
5. **Hot store projections** for API reads.
6. **Operator API + thin map UI** (tracks + health).
7. **Rule-based risk + alerts**.
8. **Approval + audit + mock integration**.
9. **Fault injector scenarios 1–12** with scorer hooks.
10. **Only then** ML detectors on critical routes with registry.
11. **Security hardening pass** before any persistent external enablement.
12. **HIL adapters** behind the same ICD-01.
13. **Field** under authorisation with limitations signed.

12.4 Configuration and secrets

- Secrets never in source control.
- Environment-specific integration endpoints; demo cannot point to production without dual control.
- Feature flags for model routes and assistant.
- Version pins for schemas and models in audit metadata.

12.5 Observability minimum viable set

Per service: /healthz, /readyz; RED metrics for APIs; consumer lag; quarantine rate; inference latency; audit write errors; handoff success/fail/DLQ depth; skew metrics. Distributed tracing across ingest→fusion→UI. Dashboards for system, model, sensor, data quality, security, operator activity.

12.6 Operator-facing epistemic rules (systems constraint on UI)

The console must visually distinguish: observation, inference, prediction, recommendation, decision. Systems engineering enforces this as a requirement because confusion here is a mishap contributor even without effectors on the platform.

12.7 Integration checklist (per adapter)

- ☐ Vendor SDK isolated in adapter process/container.
- ☐ Canonical schema mapping documented.
- ☐ Idempotency key strategy defined.
- ☐ Bounded buffer + shed policy.
- ☐ Health heartbeat.
- ☐ Contract tests green.
- ☐ Twin twin-double (sim adapter) exists for CI.
- ☐ No effector credentials present.
- ☐ Observability labels: site, sensor_id, adapter_version.

12.8 HIL bridge checklist

- ☐ Twin scenarios still pass on software path.
- ☐ Hardware adapter reuses ICD-01.
- ☐ Latency budgets remeasured; NFRs updated or waivers recorded.
- ☐ New FMEA rows reviewed.
- ☐ Still no harmful effects path from platform APIs.
- ☐ Rollback: disable hardware adapter, revert to sim, without schema break.

12.9 Field trial checklist

- ☐ Stage 6 security exit complete.
 - ☐ Authorisation package signed.
 - ☐ Limitations document acknowledged by operators and commanders.
 - ☐ On-call and degrade runbooks rehearsed.
 - ☐ External handoff disabled or mock unless separately accredited.
 - ☐ Data retention and privacy constraints applied.
 - ☐ Export-control review as required by jurisdiction.
-

13. Pre-ship gate and ownership conclusions

13.1 Pre-ship questions

Question	Answer posture
Failure modes understood?	FMEA drafted and linked to chaos; site severity pending CONOPS
Observable in production?	NFR-OBS designed; must be implemented before Stage 8
Safe rollback?	Config/model/event replay designed; drill required
Complexity proportional to value?	Challenged via modular (not nano) services and central-primary MVP
Want 3-year ownership?	Yes if interfaces remain stable and Won't-haves hold

If any answer is No without a mitigation plan, delay Stage exit.

13.2 Trade-off summary (programme level)

Decision	Gain	Loss	Harder later if wrong
Event-driven modular core	Replay, decoupling	Consistency complexity	Replacing bus after adapters proliferate
Category-only external API	Safety	Cannot “demo kinetic”	Unwinding effector coupling
Twin before HIL	Cheaper debugging	Upfront sim investment	Field failures that are really contract bugs
Central-primary hybrid	Operability	Network dependence	Rewriting schemas for edge promotion (avoidable if designed)
Pi restricted to light roles	Credibility, performance	Less cute demos	Replacing Pi “fusion” mid-programme

13.3 Residual risks

1. Silent model failure (FM-14) remains hard to detect—invest in calibration monitors and human evidence norms.
2. Fusion over-merge (FM-11) harms trust—conservative association and evidence UI are mandatory.
3. Sensor miscalibration (FM-02) — cross-sensor residuals and quarantine workflows.
4. Schedule pressure to enable real integration early — governance co-approval required.
5. Assumption A-03 (category handoff) rejected by customer — still do not put effector primitives in this platform; renegotiate boundary systems.

13.4 Conclusions

Counter-swarm defence decision-support platforms are systems engineering problems first. Algorithms, models, and UI matter, but they inherit their correctness from mission boundaries, MoSCoW honesty, ICD discipline, hybrid compute placement that matches physics and operations, twin-first verification, living FMEA, and staged exposure to hardware and field conditions. The

selected architecture—an event-driven modular service core with central-primary hybrid edge, Jetson for vision inference when needed, Pi only for light ingest, and a hard category-only external handoff—optimises for multi-year ownership and auditability under uncertainty.

The programme proves itself not by a single demo but by a dossier: traceable Must requirements, contract-tested interfaces, twin scenarios with faults, chaos mapped to FMEA, safety denials that hold under abuse, human-factors evidence, and signed limitations. That is the standard this monograph sets for Stages 0–8.

13.5 Future work (Pass 2+)

- Quantified TPM dashboards tied to each NFR with laboratory measurements.
- Expanded related-work survey on distributed fusion architectures and C-UAS system-of-systems case studies (public literature only).
- Formal mapping of every FR/NFR to automated tests as implementation proceeds.
- Site-specific CONOPS annexes (controlled distribution).
- Deeper treatment of accreditation processes where customers require formal VV&A.

14. References

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Bar-Shalom, Y., Li, X. R., & Kirubarajan, T. (2001). *Estimation with applications to tracking and navigation*. Wiley.

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1), 32–64.

Glaessgen, E. H., & Stargel, D. S. (2012). The digital twin paradigm for future NASA and U.S. Air Force vehicles. In *53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference*.

Grieves, M. (2014). Digital twin: Manufacturing excellence through virtual factory replication (White paper).

Hall, D. L., & Llinas, J. (1997). An introduction to multisensor data fusion. *Proceedings of the IEEE*, 85(1), 6–23.

IEC. (2010). *IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems*. International Electrotechnical Commission.

IEC. (2018). *IEC 60812: Failure modes and effects analysis (FMEA and FMECA)*. International Electrotechnical Commission.

IEEE. (2005). *IEEE Std 1220-2005: IEEE standard for application and management of the systems engineering process*. IEEE.

INCOSE. (2015). *Systems engineering handbook: A guide for system life cycle processes and activities* (4th ed.). Wiley.

INCOSE. (2023). *Systems engineering handbook* (5th ed.). Wiley. (Edition reference for programmes adopting the latest handbook revision.)

ISO. (2017). *ISO 10007: Quality management — Guidelines for configuration management*. International Organization for Standardization.

ISO/IEC/IEEE. (2015). *ISO/IEC/IEEE 15288:2015: Systems and software engineering — System life cycle processes*. ISO/IEC/IEEE.

ISO/IEC/IEEE. (2019). *ISO/IEC/IEEE 15289:2019: Systems and software engineering — Content of life-cycle information items (documentation)*. ISO/IEC/IEEE.

NASA. (2016). *NASA systems engineering handbook* (NASA/SP-2016-6105 Rev2). National Aeronautics and Space Administration.

Sargent, R. G. (2013). Verification and validation of simulation models. *Journal of Simulation*, 7(1), 12–24.

SEBoK Editorial Board. (ongoing). *The guide to the systems engineering body of knowledge (SEBoK)*. www.sebokwiki.org.

US Department of Defense. (2012). *MIL-STD-882E: System safety*. US DoD.

Additional programme-internal artefacts (same repository): [architecture/system.md](#), [architecture/interfaces.md](#), [architecture/failure-modes.md](#), [architecture/edge-hardware.md](#), [architecture/tradeoffs.md](#), [docs/systems/requirements.md](#), [docs/systems/digital-twin.md](#), [docs/systems/roadmap.md](#), [docs/systems/testing-strategy.md](#), [docs/systems/integration-notes.md](#).

Appendix A — Glossary

Term	Meaning
Adapter	Boundary component mapping vendor/sim data to canonical observation
Category handoff	External message expressing authorised response category, not effector primitives
Coasting	Track propagation without fresh associations; uncertainty grows
ICD	Interface Control Document
HIL	Hardware-in-the-loop
MoSCoW	Must / Should / Could / Won't prioritisation
Quarantine	Isolation of invalid messages with reason codes
RPN	Risk Priority Number (indicative)
Twin	Digital twin / simulation environment exercising production contracts
V&V	Verification and validation

Appendix B — Requirements ID index (compact)

FR-SEN-, *FR-NRM*-, FR-EVT-, *FR-DET*-, FR-CLS-, *FR-TRK*-, FR-FUS-, *FR-BEH*-, FR-RSK-, *FR-ALT*-, FR-UI-, *FR-AUD*-, FR-EXT-, *FR-SIM*-, FR-IAM-, *FR-MLG*-, FR-AIG-; *NFR-LAT*-, NFR-AVL-, *NFR-REL*-, NFR-SCL-, *NFR-SEC*-, NFR-MNT-, *NFR-TST*-, NFR-OBS-, *NFR-REC*-, NFR-XAI-*. Full text in docs/systems/requirements.md.

Appendix C — ICD ID index (compact)

ICD-01 ... ICD-15 as listed in Section 6. Full attribute tables in architecture/interfaces.md.

Appendix D — FMEA ID index (compact)

FM-01 ... FM-27 as listed in Section 9. Full living table in architecture/failure-modes.md.

Appendix E — Stage exit one-pagers (templates)

Stage 0 exit

- ☐ Mission and Won't-haves agreed
- ☐ MoSCoW agreed
- ☐ Architecture + ADRs reviewed
- ☐ ICD draft reviewed
- ☐ FMEA draft reviewed
- ☐ Twin architecture reviewed
- ☐ Roadmap and test strategy reviewed
- ☐ Open questions register acknowledged
- ☐ Approval to start Stage 1 coding

Stage 1 exit

- ☐ Compose stack runs sim→UI path
- ☐ Contract smoke for ICD-01/02/05/09
- ☐ One fault scenario demonstrated (fail visible)
- ☐ Health/ready probes present
- ☐ No effector credentials in environment

Stage 7 exit

- ☐ Twin regression still green
- ☐ HIL latency/reliability report
- ☐ FMEA hardware rows added
- ☐ Safety proofs re-run

- [] Rollback to sim verified

Stage 8 exit

- [] Authorisation package complete
- [] Limitations signed
- [] Security Stage 6 evidence attached
- [] On-call rehearsal done
- [] External handoff policy explicit (mock vs accredited)

Appendix F — Worked integration example (laboratory narrative)

This appendix walks a laboratory integration narrative that a systems engineer can rehearse with software and simulation engineers. It is deliberately concrete, still defensive, and still free of effector detail.

Hour 0–4. Freeze a minimal `observation.v1` JSON Schema: `observation_id`, `sensor_id`, `sensor_type`, `timestamp_utc`, `frame_id` (optional), geospatial fields or bearing/range depending on modality, `quality`, `schema_version`, `idempotency_key`. Publish the schema to the registry subject with BACKWARD compatibility. Write three fixtures: valid radar-like point, valid EO detection passthrough, invalid CRS.

Hour 4–12. Bring up Redpanda (or Kafka) and a normaliser that consumes adapter pushes, validates, writes quarantine on failure, and produces to observations. Metrics: `validate_fail_total`, `produce_error_total`. Prove quarantine with the invalid fixture.

Hour 12–24. Implement sim adapter that reads scenario “single UAV-like” and emits candidate observations at 10 Hz with Gaussian noise. Confirm ICD-01 latency transform p95 under 50 ms on lab hardware.

Day 2. Baseline tracker consumes detections or observations, emits `track.update.v1` with coasting after timeout. Operator API projects active tracks to Postgres. UI shows map markers with track IDs only—no risk yet.

Day 3. Inject fault: drop sensor for 30 s. Confirm D1 behaviour: coverage hole indication, no invented track in the hole. Inject duplicate burst: confirm idempotent counts.

Day 4. Rule-based risk: if track inside geofence and speed band, raise T2 alert with evidence links. Approval service records DISMISS and HEIGHTEN_MONITORING only. Mock integration receives handoff; kill mock mid-flight; confirm DLQ and local decision persistence.

Day 5. Run scenarios 3, 7, 10 from the must-have suite. Record seeds. File gaps as ICD or tracker issues—not as “UI polish.”

This narrative is the opposite of a vendor booth demo: it privileges contracts, faults, and audit over cinematic visuals.

Appendix G — Competing quality attribute scenarios

Quality attribute scenarios (Bass et al., 2012) force architecture tactics into the open.

Latency scenario. Source: EO edge camera at 30 FPS compressed detections. Stimulus: burst of 50 objects entering FOV. Artefact: detection service + bus + tracker. Environment: Stage 3 lab with Jetson edge profile. Response: track updates visible in console. Response measure: $p95 \leq 1.5$ s end-to-end after detection publish under documented load.

Safety scenario. Source: compromised or buggy recommender. Stimulus: recommendation event crafted to look like a command. Artefact: approval + integration API. Environment: staging. Response: integration rejects; audit records attempt. Response measure: 100% deny in automated suite; no network call to external mock without decision_id.

Modifiability scenario. Source: new acoustic vendor. Stimulus: add adapter. Artefact: adapter container + schema mapping. Environment: Stage 2. Response: no core service changes beyond config allowlist. Response measure: diff confined to sensor-adapter-acoustic-* and docs; contract tests added within one sprint.

Availability scenario. Source: bus broker kill. Stimulus: SIGKILL on one broker. Artefact: multi-broker lab cluster. Environment: Stage 6 staging. Response: producers retry; UI shows D3 if lag exceeds threshold; no silent empty map without banner. Response measure: recovery within RTO drill target; zero acknowledged-event loss within retention.

Appendix H — Organisational roles (RACI sketch)

Artefact	SYS	SE	ML	DE	PD	SG
Mission / MoSCoW	A/R	C	C	C	C	C
ICD	A	R	C	C	I	C
Twin architecture	A	C	C	C	C	I
FMEA	A	C	C	C	C	C
Safety proofs	A	R	C	I	I	A
Field gate	A	C	I	I	C	A
Edge hardware baseline	A	R	C	I	I	C

A = accountable, R = responsible, C = consulted, I = informed. Adjust to organisational reality; do not leave ICD without an accountable owner.

Appendix I — Extended discussion: consistency, time, and association

Multi-sensor platforms fail in subtle ways when time and identity are treated casually. Systems engineering must specify:

1. **Time domain.** All normalised events carry UTC timestamps; sensor clocks are synchronised via NTP or PTP with monitored skew. Residual skew becomes a quality flag that widens association gates rather than a silent mis-merge.
1. **Identity domain.** `observation_id` uniqueness is producer responsibility plus normaliser enforcement where possible. Track IDs are platform-issued and never reused within retention windows in ways that confuse audit.
1. **Consistency domain.** The operator console is a near-real-time projection, not a linearisable global truth. ICD must state that track views may lag and that banners communicate lag. Decisions reference evidence digests so after-action review can reconstruct what the operator saw.
1. **Association policy as configuration.** Whether the system prefers under-merge or over-merge in ambiguous cases is a policy choice with safety and operational consequences. It belongs in versioned configuration with tests, not in undocumented tribal knowledge.

These topics are where “event-driven architecture” stops being a slogan and becomes a set of explicit promises to operators and auditors.

Appendix J — Extended discussion: edge fleet operations

Introducing Jetson nodes creates a miniature fleet problem: JetPack versions, CUDA/TensorRT ABI drift, model packaging, remote health, secure update, and physical access controls. Systems engineering should:

- Treat edge software as a **deploy profile** of the same adapter interfaces.
- Pin BSPs per site release train.
- Require signed containers/models where policy demands.
- Monitor thermal throttling as a first-class health signal (throttling looks like “ML got worse”).
- Keep central capable of operating with a subset of edges offline (D1).

Raspberry Pi devices, if used, should be inventoried as non-ML gateways with clear CPU budgets and no expectation of YOLO-class throughput in summer thermal conditions.

Appendix K — Extended discussion: audit as a safety function

Audit is often treated as compliance overhead. In this platform class, audit is part of the safety and accountability story:

- It reconstructs why a human selected a category.
- It records which model versions influenced the display.
- It fails closed when write integrity cannot be guaranteed for decision commits.
- It must survive UI outage (decisions via backup channel still append).

Hash-chained append logs or WORM/immutable storage are sufficient patterns; novel consensus ledgers are unnecessary complexity for MVP.

Appendix L — Mapping to ISO/IEC/IEEE 15288 technical processes (indicative)

15288 technical process	Programme artefacts
Business or mission analysis	Executive definition; mission threads
Stakeholder needs and requirements	User list; HF inputs; MoSCoW
System requirements definition	docs/systems/requirements.md
Architecture definition	architecture/system.md, ADRs
Design definition	Service-level designs; schemas
System analysis	Tradeoff matrix; edge study
Implementation	Stage 1+ code
Integration	ICD; Compose/K8s assembly
Verification	Testing strategy; contract/sim/chaos
Transition	Stage 8→production runbooks
Validation	HF trials; MOEs; field authorised tests
Operation	SLOs; on-call; degrade modes
Maintenance	Patch trains; model rollback
Disposal	Data retention end-of-life; key destruction

This mapping is a navigation aid, not a claim of formal 15288 conformity assessment.

Appendix M — “What good looks like” versus “what demos look like”

Demos often show	Good systems look like
Perfect tracks on sunny days	Explicit uncertainty and conflict flags
Instant answers	Lag banners and coasting markers
AI chatbot centrality	Assistant optional; evidence primary
Kinetic theatre	Category handoff to accredited externals
One magical box	Adapters, ICD, twin, audit, runbooks
Pi cluster hero shot	x86 central + justified Jetson edges

Systems engineers are responsible for steering stakeholders from the left column to the right without apology.

Appendix N — Risk register (systems programme, initial)

Risk	Likelihood	Impact	Mitigation
ICD drift between agents/teams	High	High	Single ICD owner; CI contract tests
Early effector pressure	Medium	Critical	Written Won't-have; SG gate
Twin underfunded	High	High	Stage 1 exit requires fault scenarios
Edge chosen for demo optics	Medium	Medium	Hardware ADR; Pi policy
ML silent failure	Medium	High	Canary, calibration, disable routes
Bus ops skill gap	Medium	High	Training; managed Kafka-API option
HF deferred	High	High	Stage 5 exit criteria include trials

Appendix O — Document control

Field	Value
Title	Systems Engineering and Integration of Counter-Swarm Defence Decision-Support Platforms
Author	Victor.I
Status	Pass 1 — structured research monograph
Classification	Defensive decision-support research; no effector procedures
Related repo paths	docs/research/systems-engineering/, architecture/, docs/systems/

Appendix P — Configuration management and baselines

ISO 10007-style configuration management is often under-applied in software-first teams that equate “git main is green” with a configuration baseline. For counter-swarm platforms, at least four baselines matter:

1. **Requirements baseline** — MoSCoW and FR/NFR IDs with change history.
2. **Interface baseline** — ICD + schema subjects + OpenAPI majors.
3. **Design baseline** — ADRs and architecture pack revision.
4. **Product baseline** — Deployed service versions, model versions, and edge BSP/JetPack pins for a named site release.

Without (2) and (4), field incidents cannot be reconstructed: “which tracker and which EO model did the operator see?” becomes a forensic guess. Systems engineering owns the baseline *definition*; software engineering owns the automation that stamps versions into audit metadata; security/governance owns approval to promote a product baseline into an environment that can reach external systems.

Change classes.

Class	Examples	Approval
Class A	ICD-11 shape; safety interlocks; Won't-have exceptions	SYS + SG
Class B	Association policy defaults; alert thresholds	SYS + ops lead
Class C	UI copy; non-critical dashboard layout	PD + SE
Class D	Twin scenario seeds for regression	SYS + sim owner

Treating every change as Class D is how safety boundaries erode. Treating every UI tweak as Class A is how programmes stall. The classification itself is a systems artefact.

Appendix Q — CONOPS linkage and operational modes

A concept of operations (CONOPS) bridges mission definition and requirements. Even a draft CONOPS prevents architecture from floating free of use. For this platform class, operational modes should be named explicitly:

Mode	Intent	Platform behaviour highlights
Lab / twin	Develop and verify	Sim adapters only; integration mock; assistant optional
Watchfloor training	HF and procedure	Twin or recorded replay; instructor role optional; no prod handoff
Operational monitoring	Live sensors, human decisions	Full pipeline; handoff policy per site accreditation
Degraded operations	Known partial failure	D1–D6 modes; heightened logging; decision freeze if audit/IAM fail
After-action	Investigation	Replay packages; read-only; elevated audit access

CONOPS also states who may enable Operational monitoring handoff, who accepts residual FMEA risk, and what “good enough” coverage means when a sensor is offline. Open questions (air gap, vendors, power envelopes) are CONOPS inputs, not engineering guesses disguised as requirements.

Appendix R — Integration anti-patterns catalogue

The following anti-patterns recur in multi-sensor C2 and counter-UAS software programmes. Each is stated with a detection cue and a corrective move.

AP-01 Vendor core. Symptom: `radar_vendor_x` types appear in fusion service. Detection: dependency grep / architecture tests. Correction: adapter quarantine; canonical-only imports in core modules.

AP-02 DB-as-bus. Symptom: trackers poll tables; lag metrics missing. Detection: no consumer lag SLI. Correction: restore durable log; use DB as projection only.

AP-03 Demo mode with prod teeth. Symptom: environment flag flips integration URL to live. Detection: safety test “demo cannot target prod.” Correction: dual control + separate credentials + deny list in code.

AP-04 Truth on the glass. Symptom: operators see simulation ground truth during trials. Detection: HF protocol review. Correction: role-gated instructor view only.

AP-05 Alert without evidence. Symptom: T3 alerts lack observation IDs. Detection: contract test on `alert.v1`. Correction: reject alert publish without evidence links.

AP-06 Model as oracle. Symptom: UI shows single class label without score or version. Detection: NFR-XAI review. Correction: hypotheses + `model_id`/version mandatory on critical path.

AP-07 Pi hero architecture. Symptom: fusion and Postgres planned on Raspberry Pi cluster. Detection: hardware ADR review. Correction: x86 central; Pi limited to light adapters.

AP-08 Silent coast. Symptom: tracks freeze without coasting flag when sensors die. Detection: scenario 9 scorer. Correction: explicit `COASTING` + uncertainty growth.

AP-09 Assistant on the critical path. Symptom: decisions blocked when LLM endpoint is down. Detection: kill assistant in chaos drill. Correction: core console independent (D6).

AP-10 Undocumented “temporary” fields. Symptom: clients depend on `x_debug_aim` style payloads. Detection: schema diff gate. Correction: ICD change request; forbid effector-shaped fields outright.

Appendix S — Proof argument structure (claim–evidence–warrant)

Borrowing the spirit of assurance cases used in safety-critical software (without claiming a full Goal Structuring Notation programme), each major claim should be writable as:

- **Claim.** A proposition about the system (e.g., “Platform APIs cannot invoke effector primitives”).
- **Evidence.** Artefacts and test results (schema reject tests; code search; credential inventory).
- **Warrant.** Why the evidence supports the claim (ICD-11 forbids fields; integration service has no weapon SDK dependencies; CI blocks schema additions matching deny patterns).
- **Caveats.** Residual limitations (external systems are out of scope; social engineering of operators is a separate control).

Example claims for Stage 6–8 dossiers:

1. Must-priority functional requirements are implemented and verified.
2. Safety interlocks hold under abuse cases.
3. Degraded modes are fail-visible and rehearsed.
4. Twin contracts match HIL adapters.
5. Operators can distinguish inference from decision.
6. Audit reconstructs material events for a sampled incident set.
7. Rollback of model and config was drilled successfully.
8. Limitations are acknowledged by named authorities.

If a claim lacks evidence, it is aspiration—not proof.

Appendix T — Scaling and performance engineering notes

NFRs such as 2k observations/s and 500 tracks are lab planning targets. Systems engineering should insist on:

- **Load profiles** tied to scenarios (cluttered sky vs sparse).
- **Backpressure behaviour** when consumers lag (producer policies; UI D3).
- **Partitioning strategy** on the bus (site_id / sensor_id) documented in ICD.
- **Stateful service failover** notes for tracking/fusion (replay vs hot standby).
- **Object store decoupling** so raw frame loss does not stop the hot path if metadata suffices (FM-21).

Premature Kubernetes adoption does not create performance; it creates operational surface. Compose until Stage 6 hardening needs justify orchestration—unless customer constraints force earlier.

Horizontal scale applies cleanly to stateless normalisers, alert evaluators, and API replicas. Tracking and fusion need explicit state stories; pretending they are “just another microservice” is how split-brain tracks appear after failover.

Appendix U — Security and systems engineering co-ownership

Threat modelling (separate security monograph / docs/security/threat-model.md) feeds systems requirements: TLS, mTLS service identity, RBAC, rate limits, secrets management, supply-chain scanning. Systems engineering translates those into ICD auth columns, deploy profiles, and Stage 6 exit criteria. Particular co-owned topics:

- **Trust boundaries** between edge and central.
- **Allowlists** for integration endpoints.
- **Audit integrity** mechanisms.
- **Model supply chain** (who may promote weights to inference gateway).
- **Prompt injection** as a systems risk if an assistant can call tools that mutate state (mitigation: read-mostly tools; human confirmation for mutations).

Security findings that require ICD changes are Class A or B; they are not “ticket later.”

Appendix V — Maintenance, obsolescence, and disposal

15288 includes maintenance and disposal. For this platform:

- **Maintenance.** Patch trains for OS/JetPack; model recalibration cycles; schema deprecation windows (N-1 consumers).

- **Obsolescence.** Vendor SDK end-of-life handled inside adapters; core remains. Camera models that cannot meet thermal envelopes are replaced without rewriting fusion.
- **Disposal.** Secure deletion of retained media per policy; key destruction; decommission of integration credentials; archival of audit per legal hold rules.

Ignoring disposal produces lingering credentials and forgotten edge nodes—classic field failure modes that never appear in Stage 1 demos.

Appendix W — Pedagogical summary for new systems engineers joining the programme

If you join mid-programme, read in this order: mission and Won't-haves; MoSCoW; system architecture; ICD; FMEA; digital twin; roadmap; testing strategy; edge hardware. Then refuse any task that requires coding against an undocumented interface or enabling effector-shaped payloads “for a demo.” Your job is to keep the platform integrable, verifiable, and honest about uncertainty—not to win a visualisation contest.

The shortest correct description of the architecture remains:

Heterogeneous sensors enter through adapters into a canonical, versioned observation stream; modular services produce tracks, behaviour indicators, and risk assessments with evidence; humans decide among authorised categories; audit records what happened; a twin proves the contracts before hardware and field ever see the system; Jetson may run EO inference at the edge; Raspberry Pi may help with light ingest; neither replaces the x86 central brain for MVP.

Everything else is elaboration, measurement, and discipline.

Author

Victor.I